

编译器

2024年7月23日 星期二 20:36

5.0.3.500 版本的编译器将 SDK Node.js 这些内容都集成到了安装包中，不需要再另外安装

模拟器

2024年7月23日 星期二 22:31

模拟器交互

常用操作	描述
滑动屏幕	将鼠标放置屏幕上方，按住鼠标左键，在屏幕上轻扫，然后释放。
拖动项目	将鼠标放置屏幕中的项目上方，按住鼠标左键，移动项目，然后释放。
单击屏幕	将鼠标放置屏幕上方，按住鼠标左键，然后释放。
双击屏幕	将鼠标放置屏幕上方，快速双击鼠标左键，然后释放。
长按屏幕	指向屏幕上的一个项目，按下鼠标左键，保持一段时间，然后释放。
输入文字	鼠标点击输入域，随后您可以使用计算机键盘或屏幕上弹出的软键盘在模拟器中键入文字。
双指缩放	Windows系统按下Control键（macOS上的Command）会弹出一个捏合手势多触点触控界面。单击鼠标左键模拟按住双指，并释放鼠标左键模拟松开双指。鼠标充当第一个手指，穿过锚点是第二个手指。拖动光标以移动第一个点。
垂直滑动	在屏幕上打开一个垂直菜单，使用鼠标滚轮滚动菜单项。单击菜单项可进行选择。
安装应用	将本地的HAP包拖动到屏幕上即可进行安装。
上传文件	将文件拖动至模拟器屏幕上即可。



模拟器工具栏

按键	功能描述
关闭	关闭模拟器
最小化	最小化模拟器窗口
更多	打开扩展菜单
置顶	将模拟器置于所有打开窗口的顶层
左/右旋转	将设备屏幕逆时针/顺时针旋转90度
增大/减小音量	调高/低设备音量，长按可持续调高/低设备音量
截屏	生成当前屏幕的截图，并将图片保存在本地
三键导航	返回、home、最近应用
摇一摇	触发设备摇一摇操作
屏幕折叠	仅支持折叠屏：切换屏幕的折叠、展开、悬停状态
扩展菜单	
电池	打开电池模拟面板
GPS	打开GPS模拟面板
虚拟传感器	打开虚拟传感器面板
网络代理	打开网络代理面板
设置	打开设置面板。可设置模拟器主题、截屏保存路径、模拟器使用语言
关于	查看模拟器版本



模拟器和真机的差异

场景	已支持	待后续支持
应用签名	模拟器不需要签名	-
UI开发	ArkUI组件、Web、窗口管理、OpenGL ES 2.0指令、OpenGL ES 3.0指令	Vulkan
媒体	图片、播放音频、播放视频（H264格式）、录音	录像、拍照/扫码
网络与连接	WiFi、本地网络、访问互联网、设置代理	蜂窝网络
通知	系统通知、华为推送（arm版本）	华为推送（x86版本）
数据管理	用户首选项、键值型数据管理、关系型数据管理	分布式数据对象
帐号管理	华为帐号（arm版本）	华为帐号（x86版本）
DFX	HiLog、FaultLog	HiTrace、Hiperf
定位	获取位置信息、地理逆编码、花瓣地图（arm版本）	花瓣地图（x86版本）
硬件模拟	屏幕旋转、GPS、电池、传感器（步数、湿度、心率、光照强度、环境温度）、摇一摇、麦克风 折叠屏模拟器支持屏幕折叠、展开、悬停模拟	相机
安全	访问控制、安全控件、通用密钥库、加解密算法库、证书管理、asset store	指纹、人脸识别
应用测试	单元测试框架、UI测试框架	-
其他	-	更多设备模拟、自由流转、AI Kit：IAP Kit（应用内支付服务）、Camera Kit（相机服务）、Call Kit（通话服务）、Connectivity Kit（短距通信服务）、Scan Kit（统一扫码服务）、Telephony Kit（蜂窝通信服务）、Share Kit（分享服务）等

类型

2024年7月23日 星期二 22:49

基本类型: string、number、boolean等

引用类型: Object、Array、自定义类等

枚举类型: Enum

联合类型: Union

类型别名: Aliases

基本知识: 类型

HarmonyOS

引用类型: Array 、自定义类等

```
let students: Array<string> = ['Xiaoming', 'Xiaozhang', 'Xiaowang', 'Xiaoli'];
let students: string[] = ['Xiaoming', 'Xiaozhang', 'Xiaowang', 'Xiaoli'];

class User {...}
let user: User = new User();
```

基本知识: 类型

HarmonyOS

枚举类型: Enum

```
enum Color {
  Red,
  Blue,
  Green
}

let favouriteColor: Color = Color.Red;
```

声明Color的枚举类型

基本知识：类型

类型别名：Aliases 允许给一个类型取一个别名，方便理解和复用

```
type Matrix = number[][];  
type NullableObject = Object | null;
```

HarmonyOS

基本知识：空安全

一般来说，有时会在声明变量时不确定初始值。在这类情况下，通常使用联合类型包含null值

```
let name: string | null = null  
console.log(name.length.toString()); // Error Message: Cannot read property length of null
```

空安全机制的三种方式

1、使用if/else 进行判空

```
if (name != null) { /* do something */ }
```

2、使用空值合并表达式，??左边的值为null时会返回表达式右边的值

```
let name: string | null = null  
const res = name ?? '';
```

3、使用? 可选链，如果是null，运算符会返回undefined

```
let name: string | null = 'aa';  
let len = name?.length;
```

如果作用于null

函数

2024年7月23日 星期二 23:02

函数的声明和使用

HarmonyOS

函数是一组一起执行多条语句的组合，形成可重用的代码块。
通过function关键字声明要告诉编译器函数的名称、返回类型和参数以及执行的内容；

```
function 函数名 ( 参数列表 ): 返回类型 {  
    函数体  
}  
  
function printStudentsInfo(students: string[]): void {  
    for (let student of students) {  
        console.log(student);  
    }  
}  
  
printStudentInfo(['Xiaoming', 'Xiaozhang', 'Xiaowang', 'Xiaoli'])  
printStudentInfo(['Xiaoming', 'Xiaozhang', 'Xiaoli']);
```

函数的声明和使用

HarmonyOS

箭头函数 / lambda表达式 简化函数声明，通常用于需要一个简单函数的地方

```
( 参数列表 ): 返回类型 => { 函数体 }
```

箭头函数的返回类型可以省略，省略时，返回类型通过函数体推断

```
const printInfo = (name: string): void => { console.log(name) };
```

执行体只有一行的情况下可以省略花括号

```
const printInfo = (name: string) => console.log(name);  
printInfo('Xiaoming');
```

箭头函数常用于作为回调函数

```
let students: string[] = ['Xiaoming', 'Xiaozhang', 'Xiaowang', 'Xiaoli'];  
students.forEach((student: string) => console.log(student));
```

UIAbility组件

2024年7月24日 星期三 20:11

声明配置

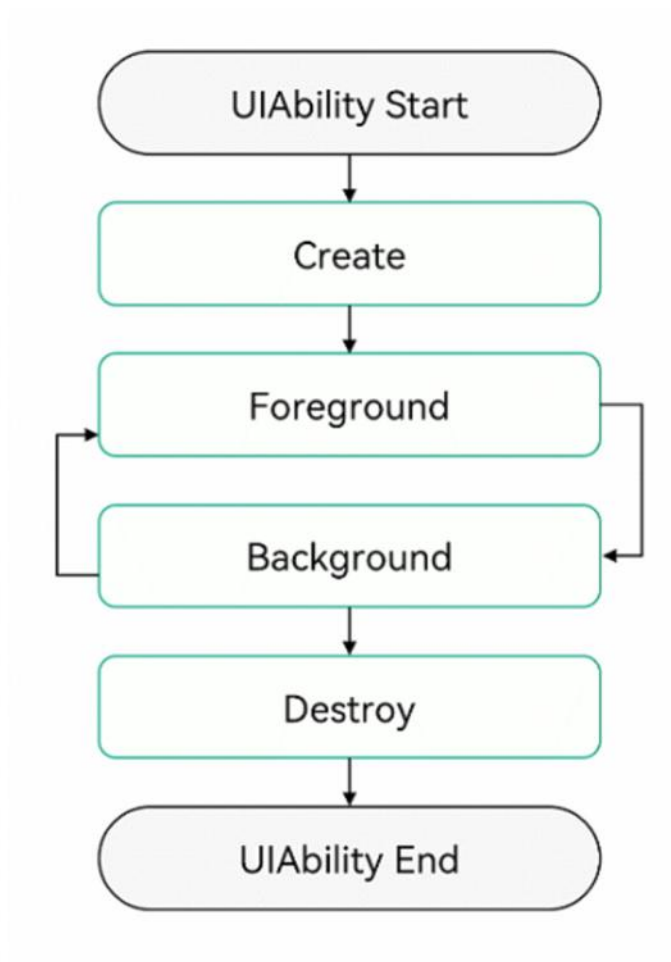
为使应用能够正常使用UIAbility，需要在`module.json5`配置文件的`abilities`标签中声明UIAbility的名称、入口、标签等相关信息。

```
{
  "module": {
    ...
    "abilities": [
      {
        "name": "EntryAbility", // UIAbility组件的名称
        "srcEntry": "./ets/entryability/EntryAbility.ets", // UIAbility组件的代码路径
        "description": "$string:EntryAbility_desc", // UIAbility组件的描述信息
        "icon": "$media:icon", // UIAbility组件的图标
        "label": "$string:EntryAbility_label", // UIAbility组件的标签
        "startWindowIcon": "$media:icon", // UIAbility组件启动页面图标资源文件的索引
        "startWindowBackground": "$color:start_window_background", // UIAbility组件启动页面背景颜色资源文件的索引
        ...
      }
    ]
  }
}
```

UIAbility生命周期

2024年7月24日 星期三 20:11

图1 UIAbility生命周期状态

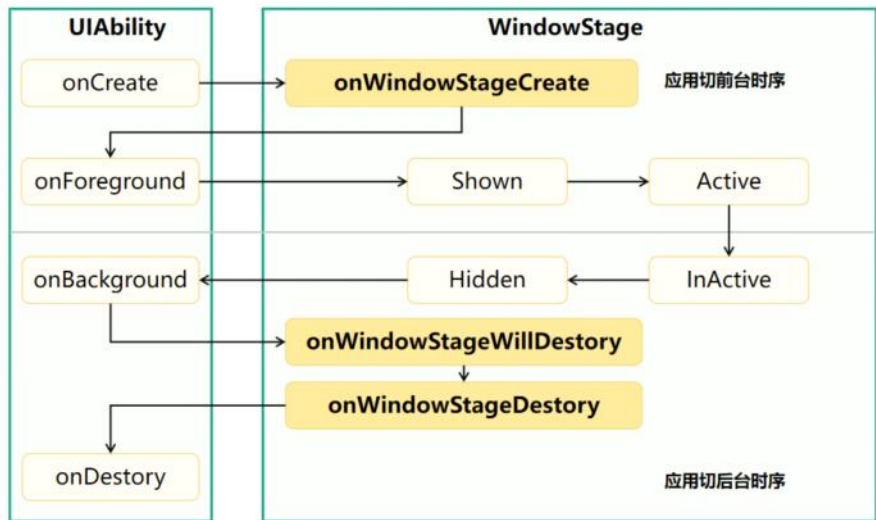


Create状态为在应用加载过程中，UA实例创建完成时触发，系统会调用onCreate（）回调

WindowStageCreate和WindowStageDestroy状态

UIAbility实例创建完成之后，在进入Foreground之前，系统会创建一个WindowStage。WindowStage创建完成后会进入onWindowStageCreate()回调，可以在该回调中设置UI加载、设置WindowStage的事件订阅。

图2 WindowStageCreate和WindowStageDestroy状态



对应于onWindowStageCreate()回调。在UIAbility实例销毁之前，则会先进入onWindowStageDestroy()回调，可以在该回调中释放UI资源。

WindowStageWillDestroy状态

对应onWindowStageWillDestroy()回调，在WindowStage销毁前执行，此时WindowStage可以使用。

Foreground和Background状态

Foreground和Background状态分别在UIAbility实例切换至前台和切换至后台时触发，对应于onForeground()回调和onBackground()回调。

onForeground()回调，在UIAbility的UI可见之前，如UIAbility切换至前台时触发。可以在onForeground()回调中申请系统需要的资源，或者重新申请在onBackground()中释放的资源。

onBackground()回调，在UIAbility的UI完全不可见之后，如UIAbility切换至后台时候触发。可以在onBackground()回调中释放UI不可见时无用的资源，或者在此回调中执行较为耗时的操作，例如状态保存等。

例如应用在使用过程中需要使用用户定位时，假设应用已获得用户的定位权限授权。在UI显示之前，可以在onForeground()回调中开启定位功能，从而获取到当前的位置信息。

当应用切换到后台状态，可以在onBackground()回调中停止定位功能，以节省系统的资源消耗。

Destroy状态

Destroy状态在UIAbility实例销毁时触发。可以在onDestory()回调中进行系统资源的释放、数据的保存等操作。

例如调用terminateSelf()方法停止当前UIAbility实例，从而完成UIAbility实例的销毁；或者用户使用最近任务列表关闭该UIAbility实例，完成UIAbility的销毁。

UIAbility组件启动模式

2024年7月24日 星期三 20:23

singleton启动模式

singleton启动模式为单实例模式，也是默认情况下的启动模式就是正常的当UA实例存在时则重用UA实例，否则新开实例

multiton启动模式

multiton启动模式为多实例模式，每次调用[startAbility\(\)](#)方法时，都会应用进程中创建一个新的该类型UIAbility实例。即在最近任务列表中可以看到有多个该类型的UIAbility实例。这种情况下可以将UIAbility配置为multiton（多实例模式）。

就是每次创建实例时都会创建新的实例，不会重用UA实例

specified启动模式

specified启动模式为指定实例模式，针对一些特殊场景使用（例如文档应用中每次新建文档希望都能新建一个文档实例，重复打开一个已保存的文档希望打开的都是同一个文档实例）。

就是说如果是创建新的文档则会打开新的实例，打开以往的文档则是旧实例

1. 在SpecifiedAbility中，将[module.json5配置文件](#)的launchType字段配置为specified。

```
{
  "module": {
    ...
    "abilities": [
      {
        "launchType": "specified",
        ...
      }
    ]
  }
}
```

在创建UA实例之前，开发者可以通过一个唯一的字符串key来为该实例指定一个启动模式。

```
// 在启动指定实例模式的UIAbility时，给每一个UIAbility实例配置一个独立的Key标识
// 例如在文档使用场景中，可以用文档路径作为Key标识
import { common, Want } from '@kit.AbilityKit';
import { hilog } from '@kit.PerformanceAnalysisKit';
import { BusinessError } from '@kit.BasicServicesKit';

const TAG: string = '[Page_StartModel]';
const DOMAIN_NUMBER: number = 0xFF00;

function getInstance(): string {
    return 'KEY';
}

@Entry
@Component
struct Page_StartModel {
    private KEY_NEW = 'KEY';

    build() {
        Row() {
            Column() {
                // ...
                Button()// ...
                .onClick(() => {
                    let context: common.UIAbilityContext = getContext(this) as common.UIAbilityContext;
                    // context为调用方UIAbility的UIAbilityContext;
                    let want: Want = {
                        deviceId: '', // deviceId为空表示本设备
                        bundleName: 'com.samples.stagemodelabilitydevelop',
                        abilityName: 'SpecifiedFirstAbility',
                        moduleName: 'entry', // moduleName非必选
                        parameters: {
                            // 自定义信息
                            instanceKey: this.KEY_NEW
                        }
                    };
                    context.startAbility(want).then(() => {
                        hilog.info(DOMAIN_NUMBER, TAG, 'Succeeded in starting SpecifiedAbility.');
```

```

let want: Want = {
  deviceId: '', // deviceId为空表示本设备
  bundleName: 'com.samples.stagemodelabilitydevelop',
  abilityName: 'SpecifiedSecondAbility',
  moduleName: 'entry', // moduleName非必选
  parameters: {
    // 自定义信息
    instanceKey: getInstance()
  }
};
context.startAbility(want).then(() => {
  hilog.info(DOMAIN_NUMBER, TAG, 'Succeeded in starting SpecifiedAbility.');
```

}).catch((err: BusinessError) => {
 hilog.error(DOMAIN_NUMBER, TAG, `Failed to start SpecifiedAbility. Code is \${err.code}, message is \${err.message}`);
})
this.KEY_NEW = this.KEY_NEW + 'a';
})
// ...
}
.width('100%')
}
.height('100%')
}
}

3. 由于SpecifiedAbility的启动模式被配置为指定实例启动模式，因此在SpecifiedAbility启动之前，会先进入对应的AbilityStage的onAcceptWant()生命周期回调中，以获取该UIAbility实例的Key值。然后系统会自动匹配，如果存在与该UIAbility实例匹配的Key，则会启动与之绑定的UIAbility实例，并进入该UIAbility实例的onNewWant()回调函数；否则会创建一个新的UIAbility实例，并进入该UIAbility实例的onCreate()回调函数和onWindowStageCreate()回调函数。

示例代码中，通过实现onAcceptWant()生命周期回调函数，解析传入的want参数，获取自定义参数instanceKey。业务逻辑会根据这个参数返回一个字符串Key，用于标识当前UIAbility实例。如果返回的Key已经对应一个已启动的UIAbility实例，系统会将该UIAbility实例拉回前台并获焦，而不会创建新的实例。如果返回的Key没有对应已启动的UIAbility实例，则系统会创建新的UIAbility实例并启动。

```

import { AbilityStage, Want } from '@kit.AbilityKit';

export default class MyAbilityStage extends AbilityStage {
  onAcceptWant(want: Want): string {
    // 在被调用方的AbilityStage中，针对启动模式为specified的UIAbility返回一个UIAbility实例对应的一个Key值
    // 当前示例指的是module1 Module的SpecifiedAbility
    if (want.abilityName === 'SpecifiedFirstAbility' || want.abilityName === 'SpecifiedSecondAbility') {
      // 返回的字符串Key标识为自定义拼接的字符串内容
      if (want.parameters) {
        return `SpecifiedAbilityInstance_${want.parameters.instanceKey}`;
      }
    }
    // ...
    return 'MyAbilityStage';
  }
}

```


UIAbility组件基本用法

2024年7月24日 星期三 21:20

UIAbility组件的基本用法包括：指定UIAbility的启动页面以及获取UIAbility的上下文 [UIAbilityContext](#)。

指定UIAbility的启动页面

应用中的UIAbility在启动过程中，需要指定启动页面，否则应用启动后会因为没有默认加载页面而导致白屏。可以在UIAbility的onWindowStageCreate()生命周期回调中，通过[WindowStage](#)对象的loadContent()方法设置启动页面。

```
import { UIAbility } from '@kit.AbilityKit';
import { window } from '@kit.ArkUI';

export default class EntryAbility extends UIAbility {
  onWindowStageCreate(windowStage: window.WindowStage): void {
    // Main window is created, set main page for this ability
    windowStage.loadContent('pages/Index', (err, data) => {
      // ...
    });
  }
  // ...
}
```

获取UIAbility的上下文信息

UIAbility类拥有自身的上下文信息，该信息为[UIAbilityContext](#)类的实例，[UIAbilityContext](#)类拥有abilityInfo、currentHapModuleInfo等属性。

通过UIAbilityContext可以获取UIAbility的相关配置信息，如包代码路径、Bundle名称、Ability名称和应用程序需要的环境状态等属性信息，以及可以获取操作UIAbility实例的方法（如startAbility()、connectServiceExtensionAbility()、terminateSelf()等）。

如果需要在页面中获得当前Ability的Context，可调用[getContext](#)接口获取当前页面关联的UIAbilityContext或ExtensionContext。

- 在UIAbility中可以通过this.context获取UIAbility实例的上下文信息。

```
import { UIAbility, AbilityConstant, Want } from '@kit.AbilityKit';

export default class EntryAbility extends UIAbility {
  onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
    // 获取UIAbility实例的上下文
    let context = this.context;
    // ...
  }
}
```

- 在页面中获取UIAbility实例的上下文信息，包括导入依赖资源context模块和在组件中定义一个context变量两个部分。

```
import { common, Want } from '@kit.AbilityKit';

@Entry
@Component
struct Page_EventHub {
  private context = getContext(this) as common.UIAbilityContext;

  startAbilityTest(): void {
    let want: Want = {
      // Want参数信息
    };
    this.context.startAbility(want);
  }

  // 页面展示
  build() {
    // ...
  }
}
```

也可以在导入依赖资源context模块后，在具体使用UIAbilityContext前进行变量定义。

```
import { common, Want } from '@kit.AbilityKit';

@Entry
@Component
struct Page_UIAbilityComponentsBasicUsage {
  startAbilityTest(): void {
    let context = getContext(this) as common.UIAbilityContext;
    let want: Want = {
      // Want参数信息
    };
    context.startAbility(want);
  }

  // 页面展示
  build() {
    // ...
  }
}
```

UIAbility组件与UI的数据同步

2024年7月24日 星期三 21:46

使用EventHub进行数据通信

[EventHub](#)为UIAbility组件提供了事件机制，使它们能够进行订阅、取消订阅和触发事件等数据通信能力。

在[基类Context](#)中，提供了EventHub对象，可用于在UIAbility组件实例内通信。使用EventHub实现UIAbility与UI之间的数据通信需要先获取EventHub对象，本章节将以此为例进行说明。

1. 在UIAbility中调用[eventHub.on\(\)](#)方法注册一个自定义事件“event1”，[eventHub.on\(\)](#)有如下两种调用方式，使用其中一种即可。

```
import { hilog } from '@kit.PerformanceAnalysisKit';
import { UIAbility, Context, Want, AbilityConstant } from '@kit.AbilityKit';

const DOMAIN_NUMBER: number = 0xFF00;
const TAG: string = '[EventAbility]';

export default class EntryAbility extends UIAbility {
  onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
    // 获取eventHub
    let eventhub = this.context.eventHub;
    // 执行订阅操作
    eventhub.on('event1', this.eventFunc);
    eventhub.on('event1', (data: string) => {
      // 触发事件，完成相应的业务操作
    });
    hilog.info(DOMAIN_NUMBER, TAG, '%{public}s', 'Ability onCreate');
  }

  // ...
  eventFunc(argOne: Context, argTwo: Context): void {
    hilog.info(DOMAIN_NUMBER, TAG, '1. ' + `${argOne}, ${argTwo}`);
    return;
  }
}
```

在UI中通过[eventHub.emit\(\)](#)方法触发该事件，在触发事件的同时，根据需要传入参数信息。

```
import { common } from '@kit.AbilityKit';
import { promptAction } from '@kit.ArkUI';

@Entry
@Component
struct Page_EventHub {
  private context = getContext(this) as common.UIAbilityContext;

  eventHubFunc(): void {
    // 不带参数触发自定义“event1”事件
    this.context.eventHub.emit('event1');
    // 带1个参数触发自定义“event1”事件
    this.context.eventHub.emit('event1', 1);
    // 带2个参数触发自定义“event1”事件
    this.context.eventHub.emit('event1', 2, 'test');
    // 开发者可以根据实际的业务场景设计事件传递的参数
  }
}
```

4. 在自定义事件“event1”使用完成后，可以根据需要调用`eventHub.off()`方法取消该事件的订阅。

```
import { UIAbility } from '@kit.AbilityKit';

export default class EntryAbility extends UIAbility {
  // ...
  onDestroy(): void {
    this.context.eventHub.off('event1');
  }
}
```


UIAbility组件间交互（设备内）

2024年7月24日 星期三 22:27

这个没学明白，不理解使用场景

Position

2024年7月26日 星期五 14:11

绝对定位是按照父组件的左上角开始定位的

参数: {x: 水平偏量值, y: 垂直偏量值}

使用定位属性的元素不参与元素的排序, 但会有层级问题, 可以通过使用 `zIndex` () 属性来改变层级

层叠布局

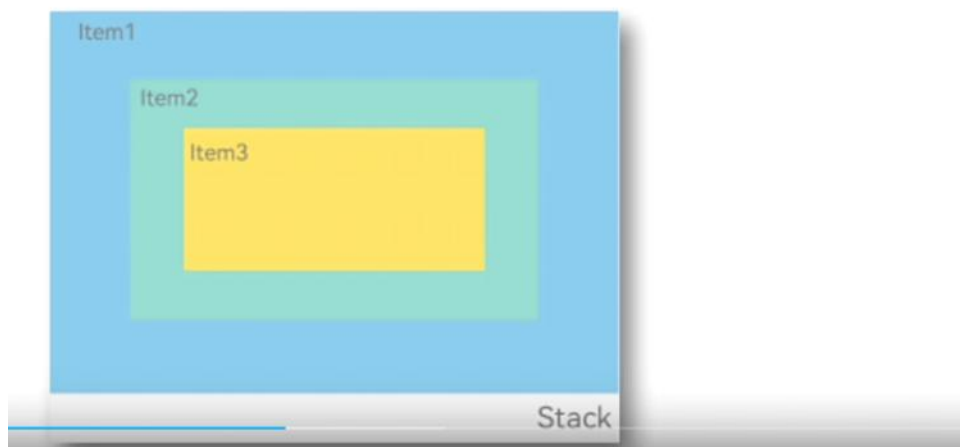
2024年7月26日 星期五 14:16

层叠布局

层叠布局具有较强的**组件层叠**能力。场景：**卡片层叠效果**等

特点：层叠操作**更简洁**，**编码效率高**。（绝对定位的优势是**更灵活**）

Stack 容器内的子元素的顺序为 Item1 -> Item2 -> Item3



extend继承和super关键字

2024年7月26日 星期五 15:30



使用extend继承父类的时候要先抽离拥有共性的属性和方法

可以使用super关键字来直接调用父类的构造函数和方法

Instanceof和typeof

2024年7月26日 星期五 15:34

typeof 只能判断简单类型

Instanceof 可以判断复杂类型

```
13
14  class Person {}
15
16  class Student extends Person {}
17
18  let s: Student = new Student()
19  console.log('判断结果:', s instanceof Student)
20  console.log('判断结果:', s instanceof Person)
21
22  @Entry
23  @Component
24  struct Index {
```

修饰符

2024年7月26日 星期五 15:40

Readonly

使用readonly表示只读不能 修改

Private

使用 private 表示该属性或方法只能在本类中使用，外部无法获取

Class 类 - 修饰符 - private

private 修饰的成员不能在声明该成员类之外访问, 包括子类

```
class 类{  
  private 属性:类型  
  private 方法(){}  
}
```

```
class Person {  
  private name: string = ''  
  private age: number = 0  
  
  sayHi() {  
    // 内部可以访问  
    console.log(`你好, 我叫:${this.name}`)  
  }  
}  
  
class Student extends Person{  
  sayHello() {  
    // 无法访问 报错  
    console.log(`你好, 我叫:${super.name}`)  
  }  
}  
  
const p = new Person()  
// p.name // 无法访问 报错  
p.sayHi()
```

Protected

使用 protected 的属性或者方法只能在本类或子类中使用，无法在外部通过实例访问

Class 类 - 修饰符 - protected

protected修饰符的作用与private修饰符非常相似

不同点是protected修饰的成员允许在 派生类(子类) 中访问

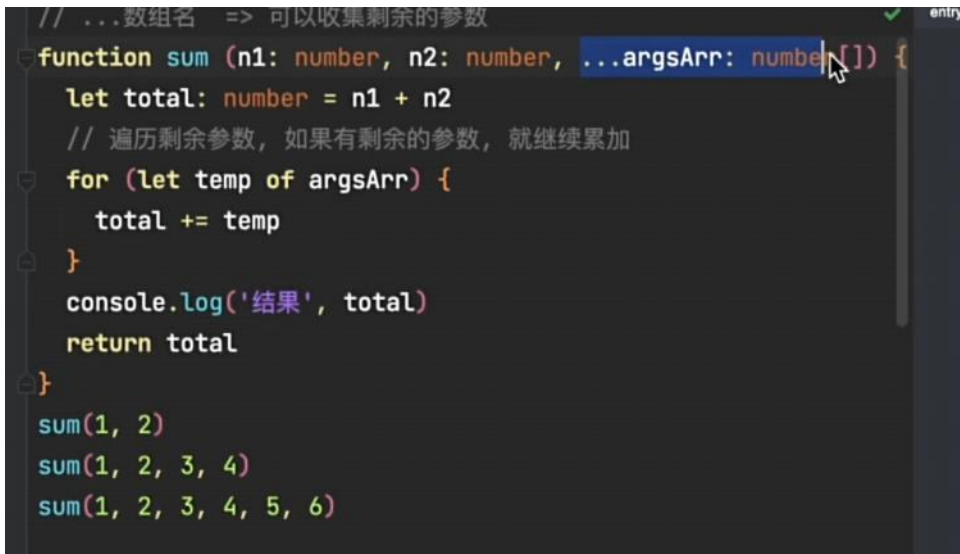
```
class 类{  
  protected 属性:类型  
  protected 方法(){}  
}
```

```
class Person {  
  protected name: string = ''  
  protected age: number = 0  
  
  sayHi() {  
    // 内部可以访问  
    console.log(`你好, 我叫:${this.name}`)  
  }  
}  
  
class Student extends Person{  
  sayHello() {  
    // 可以访问到父类的protected属性  
    console.log(`你好, 我叫:${super.name}`)  
  }  
}  
  
const p = new Person()  
// p.name // 无法访问 报错  
p.sayHi()
```

Public

使用public在哪都能用

剩余参数可以在某个方法需要传入多个参数时使用

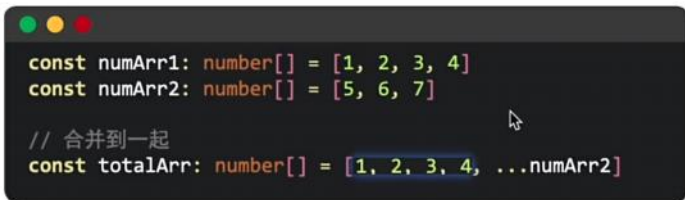


```
// ...数组名 => 可以收集剩余的参数
function sum (n1: number, n2: number, ...argsArr: number[]) {
  let total: number = n1 + n2
  // 遍历剩余参数，如果有剩余的参数，就继续累加
  for (let temp of argsArr) {
    total += temp
  }
  console.log('结果', total)
  return total
}

sum(1, 2)
sum(1, 2, 3, 4)
sum(1, 2, 3, 4, 5, 6)
```

展开运算符在 ArkTs 中只能用在数组上

出于程序稳定性，以及运行性能考虑，在 ArkTS 中 ...**(展开运算符)** 只能用在数组上

A code editor window with a dark background and three colored window control buttons (green, yellow, red) in the top-left corner. It contains three lines of TypeScript code. The first line is 'const numArr1: number[] = [1, 2, 3, 4]'. The second line is 'const numArr2: number[] = [5, 6, 7]'. The third line is '// 合并到一起' followed by 'const totalArr: number[] = [1, 2, 3, 4, ...numArr2]'. The '1, 2, 3, 4' part of the third line is highlighted with a blue selection bar.

```
const numArr1: number[] = [1, 2, 3, 4]
const numArr2: number[] = [5, 6, 7]
// 合并到一起
const totalArr: number[] = [1, 2, 3, 4, ...numArr2]
```


继承

2024年7月26日 星期五 15:52

接口补充 – 接口继承

接口继承使用的关键字是 `extends`

```
interface 接口1 {  
  属性1: 类型  
}  
interface 接口2 extends 接口1 {  
  属性2: 类型  
}
```

```
interface IAnimal {  
  name: string  
}  
interface ICat extends IAnimal {  
  color: string  
}  
  
const cat: ICat = {  
  name: '加菲猫',  
  color: '黑色'  
}
```

接口实现

2024年7月26日 星期五 15:57

定义一个接口来约束类，类需要按照接口的要求实现类的主体

泛型函数

2024年7月26日 星期五 16:02

泛型函数可以让函数的返回参数和接受参数可变，更加的灵活

```
// 封装了一个函数：传入什么样的参数，就立刻返回什么样的参数
function fn<T> (param: T) : T {
    return param
}

fn<string>('abc')
fn<number>(123)
fn<boolean>(true)
fn<number[]>([1, 2, 3, 4, 5])

// 会默认根据传参，进行类型推断，动态的配置 T 类型参数 的值
fn(true)
fn([1, 2, 3, 4, 5])
```

同时 ArtTs 会根据传递的参数类型来判断函数的类型

```
// 练习1：定义函数，参数是数组(存的类型不定)，返回数组的长度
function getLength<T> (arr: T[]) : number {
    return arr.length
}

console.log('', getLength<number>([1, 2, 3]))
console.log('', getLength<string>(['1', 'aa', 'bb']))
```

多个泛型参数直接使用逗号进行分隔即可

泛型接口

2024年7月26日 星期五 16:26

```
5
6 interface IdFunc<T>{
7     id: (value: T) => T
8     ids: () => T[]
9 }
10
11 let obj: IdFunc<number> = {
12     id(value: number){
13         return value
14     },
15     ids(){
16         return[1, 2, 3]
17     }
18 }
19
20
21
```

泛型类

2024年7月26日 星期五 16:33

定义类的时候，结合泛型定义，就是泛型类。

```
class 类名<Type>{  
    // 内部可以使用 Type  
}
```

```
// 定义  
class Person <T> {  
    id: T  
  
    constructor(id: T) {  
        this.id = id  
    }  
    getId(): T {  
        return this.id  
    }  
}  
// 使用  
let p = new Person<number>(10)
```

泛型约束

2024年7月26日 星期五 16:16

给传递的类型参数添加约束

```
function fn<T> (arr: T){  
    console.log(arr.length)  
}  
  
fn(true)
```

上面的代码中并不是所有的类型都有length属性，这种情况就可以进行约束
约束后下面则是更加严谨的写法

```
interface Ilength{  
    length: number  
}  
  
function fn<T extends Ilength> (arr: T){  
    console.log("Fjccc", arr.length)  
}  
  
fn(true)
```